



Université Moulay Ismail

Ecole Normale Supérieure d'Arts et Métiers de Meknès

FILIERE : GIATD-SI

*IMPLÉMENTATION COMPLÈTE DE L'ALGORITHME QUEST
EN PYTHON*

Encadrant : Mr Hosni Mohamed

RÉALISÉ PAR :

Sohaib Daoudi

Souleymane Diallo

Hinimdou Morsia Guitdam

Résumé

Les arbres de classification basés sur des algorithmes de recherche exhaustive ont tendance à être biaisés en faveur de la sélection de variables offrant un grand nombre de possibilités de découpe. En conséquence, de tels arbres doivent être interprétés avec prudence.

Ce rapport présente un algorithme appelé **QUEST** (*Quick, Unbiased, Efficient Statistical Tree*) qui présente un biais négligeable dans la sélection des variables de partitionnement. Sa stratégie de sélection des coupures partage certaines similitudes avec la méthode **FACT**, tout en produisant exclusivement des coupures binaires. L'arbre final peut être obtenu soit par l'application d'une règle d'arrêt directe, soit par une procédure d'élagage (*pruning*).

Des jeux de données réels et simulés sont utilisés afin de comparer la méthode QUEST à l'approche classique de recherche exhaustive. Les résultats montrent que QUEST est nettement plus rapide et que la taille ainsi que la précision de classification des arbres générés sont généralement comparables à celles des arbres issus des méthodes exhaustives.

Mots-clés : Arbres de décision, analyse discriminante, apprentissage automatique.

Historique de la méthode QUEST

Les premières méthodes de construction d'arbres de classification, telles que **THAID** (années 1960) et **CART** (Breiman et al., 1984), reposaient sur une recherche exhaustive de toutes les coupures possibles afin de sélectionner celle minimisant une mesure d'impureté. Bien que efficaces, ces approches se sont révélées biaisées en faveur des variables possédant un grand nombre de valeurs distinctes, en particulier les variables continues ou catégorielles à forte cardinalité.

C'est dans ce contexte que la méthode **QUEST** a été proposée à la fin des années 1990 par **Wei-Yin Loh** et **Yu-Shan Shih**. L'objectif principal de cette méthode était de corriger le biais de sélection des variables tout en réduisant le coût computationnel des algorithmes classiques.

QUEST introduit une approche statistique basée sur des tests de significativité pour sélectionner la variable de séparation avant de déterminer le point de coupure optimal. Cette stratégie permet de dissocier le choix de la variable de celui de la coupure, garantissant ainsi une sélection plus équitable et plus rapide.

Grâce à ces caractéristiques, la méthode QUEST s'est imposée comme une alternative efficace et fiable aux méthodes traditionnelles de construction d'arbres de classification, en particulier dans les contextes impliquant des jeux de données de grande dimension.

1 Construction et fonctionnement des arbres de classification

Un arbre de classification permet de prédire la classe d'un objet à partir de ses variables prédictives.

L'arbre est construit récursivement, en partitionnant l'échantillon d'apprentissage à chaque nœud.

Chaque nœud correspond à une partition des données.

Pour chaque variable :

- **Variable ordonnée** → on teste toutes les valeurs possibles pour trouver la coupure qui minimise l'impureté (ex. indice de Gini).
- **Variable catégorielle** → on teste toutes les sous-parties possibles pour séparer les classes.

Exemple Iris : l'arbre coupe d'abord selon la longueur du pétale, puis la largeur, avec un taux d'erreur apparent de 4 % et une estimation jackknife de 5 % ± 2 %.

2 Limites et problèmes des méthodes classiques (recherche exhaustive)

Complexité computationnelle :

- Pour une variable ordonnée, le nombre de coupures à tester est linéaire dans le nombre de valeurs.
- Pour une variable catégorielle, il est exponentiel selon le nombre de catégories.
- Des coupures plus flexibles peuvent être obtenues en combinant plusieurs variables dans une **coupure en combinaison linéaire** de la forme :

$$\sum_{k=1}^K a_k x_k \leq c$$

La recherche exhaustive de ces combinaisons exige d'évaluer la fonction d'impureté pour chaque ensemble $\{a_1, \dots, a_K, c\}$, ce qui est beaucoup plus difficile et peut présenter plusieurs **maxima locaux** de la fonction objective.

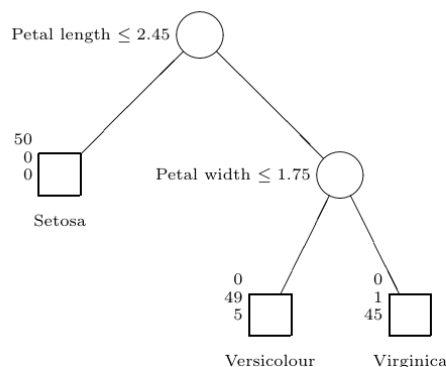


Figure 1: Figure 1 : Données Iris analysées avec recherche exhaustive, indice de Gini, 10-fold CV pruning et règle 1-SE. Le triple à côté de chaque nœud terminal indique le nombre de cas de Setosa, Versicolour et Virginica dans le nœud. Estimation jackknife de l'erreur de classification : 0.053 ± 0.018.

Biais dans la sélection de variable :

- Les variables offrant plus de possibilités de coupures sont souvent privilégiées, ce qui rend l'interprétation de l'arbre moins fiable.

Ces problèmes ont conduit à rechercher des solutions ****heuristiques ou approchées****, plus rapides et moins biaisées.

3 Méthodes alternatives : FACT et QUEST

FACT :

- Sépare la sélection de variable et la sélection du point de coupure pour simplifier le calcul.
- Utilise une ANOVA F pour sélectionner la variable la plus pertinente.
- Applique ensuite LDA pour déterminer le meilleur point de coupure.
- Les variables catégorielles sont transformées en ordonnées.

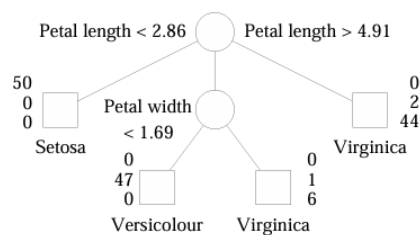


Figure 2 : Données Iris analysées avec la méthode FACT. Le triple à côté de chaque nœud terminal indique le nombre de cas de Setosa, Versicolour et Virginica, respectivement, dans le nœud. L'estimation jackknife de l'erreur de classification est de $0,033 \pm 0,015$.

La Figure 2 montre l'arbre FACT pour les données Iris. Il utilise les mêmes deux variables que dans la méthode de recherche exhaustive. La première coupure produit trois nœuds car il y a trois classes. La deuxième coupure, en revanche, ne produit que deux nœuds, car seules deux classes restent dans les données. L'arbre classe incorrectement trois des échantillons d'apprentissage et l'estimation jackknife de l'erreur est de $3 \% \pm 2 \%$.

QUEST :

FACT a été développé pour simplifier le calcul et corriger partiellement le biais des méthodes de recherche exhaustive. Il sépare la sélection de la variable et la détermination du point de coupure, mais il n'élimine pas totalement le biais lorsque certaines variables sont catégorielles. De plus, il utilise une règle d'arrêt direct, ce qui peut le rendre moins performant que des méthodes avec élagage « bottom-up » comme CART dans certaines applications.

- Corrige le biais de sélection des variables dans FACT.
- Permet des arbres rapides et stables, comparables à la recherche exhaustive en précision, mais avec un coût computationnel réduit.
- Estimation jackknife pour le taux d'erreur comparable ou légèrement supérieur à FACT, mais les différences ne sont pas significatives.

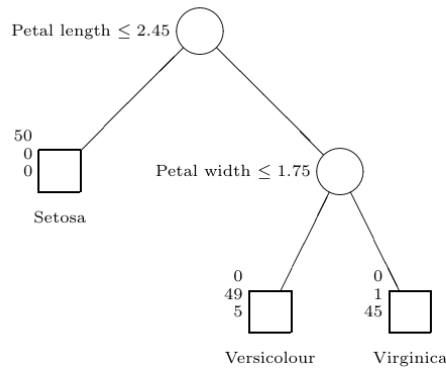


Figure 3 : Données Iris analysées avec la méthode QUEST, 10-fold CV pruning et règle 1-SE. Le triple à côté de chaque nœud terminal indique le nombre de cas de Setosa, Versicolour et Virginica dans le nœud. Estimation jackknife de l'erreur de classification : $0,040 \pm 0,016$.

QUEST : Étapes Mathématiques Complètes

1. Sélection du point de coupure (Split Point Selection)

1.1 Variables ordonnées

Cas $J = 2$ classes : On utilise une version modifiée de l'Analyse Discriminante Quadratique (QDA).

Notations : Pour la classe $j = 1, 2$:

$$\bar{x}^{(j)} = \text{moyennedelaclasse } j, \quad s_j^2 = \text{variancedelaclasse } j$$

$$p(j|t) = \text{proportiondelaclass } j \text{ dans le nœud } t$$

Équation QDA pour les densités :

$$p(1|t) \frac{1}{s_1} \phi\left(\frac{x - \bar{x}^{(1)}}{s_1}\right) = p(2|t) \frac{1}{s_2} \phi\left(\frac{x - \bar{x}^{(2)}}{s_2}\right)$$

où $\phi(z) = (2\pi)^{-1/2} \exp(-z^2/2)$ est la densité normale standard.

Transformation en équation quadratique :

$$ax^2 + bx + c = 0$$

avec :

$$a = s_1^2 - s_2^2, \quad b = 2(\bar{x}^{(2)} s_1^2 - \bar{x}^{(1)} s_2^2), \quad c = (\bar{x}^{(1)} s_2)^2 - (\bar{x}^{(2)} s_1)^2 + 2s_1^2 s_2^2 \log \frac{p(2|t) s_1}{p(1|t) s_2}$$

Choix de la racine :

$$d^* = \arg \min_{d \in \{d_1, d_2\}} \left| d - \frac{\bar{x}^{(1)} + \bar{x}^{(2)}}{2} \right|$$

Cas $J > 2$ classes :

1. Regrouper les J moyennes de classe $\bar{x}^{(1)}, \dots, \bar{x}^{(J)}$ en 2 super-classes via un 2-means clustering.
2. Calculer les statistiques des super-classes : $\bar{x}_A, s_A^2, \bar{x}_B, s_B^2$, et $p(A|t), p(B|t)$.
3. Résoudre la même équation quadratique pour obtenir le split.
4. Règle de choix : si les deux racines donnent des nœuds non vides, choisir celle du milieu ; sinon, $d = (\bar{x}_A + \bar{x}_B)/2$. Si échec, prendre la médiane des valeurs uniques.

1.2 Variables catégorielles

Transformation en variable ordonnée (CRIMCOORD)

Étapes :

1. Codage dummy pour M modalités $c_1, \dots, c_M$:

$$v_l = \begin{cases} 1 & \text{si } X = c_l \\ 0 & \text{sinon,} \end{cases} \quad l = 1, \dots, M$$

2. Calcul des matrices de dispersion :

$$B = \sum_{j=1}^J N_j (\bar{v}^{(j)} - \bar{v})(\bar{v}^{(j)} - \bar{v})^T, \quad W = \sum_{j=1}^J \sum_{i=1}^{N_j} (v_i^{(j)} - \bar{v}^{(j)})(v_i^{(j)} - \bar{v}^{(j)})^T$$

$$T = B + W$$

3. Analyse discriminante : résoudre $T^{-1}Ba = \lambda a$, prendre le vecteur propre principal.
4. Transformation des données :

$$\xi = a^T v$$

puis appliquer l'algorithme de split sur ξ comme pour les variables ordonnées.

2. Sélection de la variable (Variable Selection)

Algorithme en deux étapes :

1. **Stage I : Tests statistiques initiaux**

- Variables ordonnées : test F d'ANOVA

$$F = \frac{\text{variance entre classes}}{\text{variance intra-classe}}$$

- Variables catégorielles : test du χ^2 d'indépendance

$$\chi^2 = \sum \frac{(O_{ij} - E_{ij})^2}{E_{ij}}$$

2. **Stage II : Correction pour variances inégales** Si aucune variable significative au Stage I :

- Variables ordonnées : test de Levene sur les déviations absolues

$$F_{Levene} = \frac{\text{variancedesdviationsabsoluesentreclasses}}{\text{variancedesdviationsabsoluesintra-classe}}$$

3. Correction Bonferroni : Soit K variables ordonnées et K_1 variables catégorielles :

$$\alpha^* = \frac{\alpha}{K + K_1}$$

4. Sélection finale :

- Calculer la p-value $\hat{\alpha}_1$ pour chaque variable ordonnée et $\hat{\beta}(k)$ pour chaque variable catégorielle.
- Sélectionner la variable avec la plus petite p-value si $\min(\hat{\alpha}_1, \hat{\beta}(k)) < \alpha^*$.
- Sinon, recalculer avec le test de Levene et sélectionner selon $\alpha/(K + K_1)$.

Étapes d'implémentation

Arbre de décision QUEST - Configuration

=====

ALPHA (niveau de signification) : 0.05

MIN_{SAMPLES}PLIT : 10

MIN_{SAMPLES}LEAF : 5

MAX_{DEPTH} : 20(limitedescurit)

DEFAULT_{CP}ALPHA : 0.0

=====

Remarque : QUEST utilise la taille de complexité des coûts (cost-complexity pruning) comme méthode principale de régularisation, et non pas max_depth ou min_samples.

Bloc B — Transformation des variables catégorielles (Algorithme 2 : CRIMCOORDS)

Lorsqu'une variable catégorielle est sélectionnée pour la division, il est nécessaire de la convertir en valeur numérique afin de déterminer un point de coupure. QUEST utilise ****CRIMCOORDS**** (CRITICAL Mean COORDinateS) à cet effet.

Pourquoi CRIMCOORDS ?

L'approche naïve de codage "one-hot" entraîne un problème en haute dimension. CRIMCOORDS fournit une ****projection 1D optimale**** en suivant ces étapes :

1. Codage one-hot des catégories $\rightarrow$ matrice V
2. Centrage de V (soustraction des moyennes des colonnes)
3. Calcul de la SVD pour gérer la déficience de rang
4. Application de l'Analyse Discriminante Linéaire (LDA) sur l'espace réduit
5. Calcul d'un score ξ pour chaque catégorie

Détails mathématiques

Soit C le nombre de catégories. La matrice one-hot V est de dimension $n \times C$. Après centrage :

$$\tilde{V} = V - \mathbf{1}_n \bar{v}^T$$

La SVD donne : $\tilde{V} = U\Sigma W^T$

Le rang $r = C - 1$ (à cause du centrage). On travaille dans l'espace réduit de dimension r .

La LDA trouve la direction qui maximise la séparation des classes :

$$a = S_W^{-1}(\bar{x}_1 - \bar{x}_2)$$

Les scores des catégories sont ensuite :

$$\xi = W_r a$$

Bloc C — Calcul du point de coupure (Algorithme 1 : Split QDA)

Une fois une variable sélectionnée, QUEST utilise l'**Analyse Discriminante Quadratique (QDA)** pour trouver le point de coupure optimal.

Pourquoi QDA ? Contrairement à la recherche exhaustive (comme dans CART), QDA fournit une **solution analytique** pour le point de coupure, rendant QUEST plus efficace computationnellement.

Formulation mathématique Supposons que la variable sélectionnée suive des distributions normales différentes dans chaque classe :

- Classe 0 : $X \sim N(\mu_0, \sigma_0^2)$ - Classe 1 : $X \sim N(\mu_1, \sigma_1^2)$

Le point de coupure QDA est celui où les probabilités a posteriori sont égales, ce qui conduit à une équation quadratique :

$$At^2 + Bt + C = 0$$

On choisit la racine physiquement significative (celle située entre les moyennes des classes).

Bloc D — Élagage Cost-Complexity

QUEST utilise le **post-élagage** via le Minimal Cost-Complexity Pruning (même méthode que CART).

Le critère de complexité pour un sous-arbre T est :

$$R_\alpha(T) = R(T) + \alpha|T|$$

où : - $R(T)$ est le taux de mauvaise classification (risque empirique) - $|T|$ est le nombre de nœuds terminaux - $\alpha \geq 0$ est le paramètre de complexité

Pour chaque nœud interne t , on calcule l'alpha effectif :

$$\alpha_{eff}(t) = \frac{R(t) - R(T_t)}{|T_t| - 1}$$

Les nœuds avec le plus petit α_{eff} sont élagués en premier, générant une séquence de sous-arbres.

4. Tests et démonstration de la méthode QUEST

Cette section vise à valider expérimentalement l'implémentation de l'algorithme QUEST à l'aide de jeux de données synthétiques et réels. Les objectifs principaux sont les suivants :

- Vérifier la compatibilité avec la classification binaire.
- Valider la gestion du cas multiclassés via le regroupement en super-classes.
- Illustrer l'effet de l'élagage *cost-complexity*.
- Évaluer les performances sur des jeux de données de référence.

Les expériences réalisées couvrent les cas suivants :

1. Classification binaire (jeu de données synthétique)
2. Classification multiclassés ($K = 4$) avec regroupement en super-classes
3. Élagage post-apprentissage par *cost-complexity pruning*
4. Jeu de données **Iris**
5. Jeu de données **UCI Car Evaluation**

Résultats expérimentaux

Synthetic Dataset Created :

- n_samples : 200

- n_features : 4
- Feature 1 : Continue (moyennes différentes)
- Feature 2 : Continue (variances différentes)
- Feature 3 : Catégorielle (4 catégories)
- Feature 4 : Bruit (non pertinente)

Premières 5 observations :

X	y
[1.61, 0.46, D, 1.88]	0
[4.63, 0.20, B, -1.35]	1
[8.29, -2.43, C, 1.27]	1
[2.38, -0.54, B, -0.42]	0
[2.50, 0.18, B, 0.34]	0

Entraînement et structure de l'arbre QUEST

Training QUEST Decision Tree...

Training complete!

QUEST Decision Tree Structure:

Classes: [np.int64(0), np.int64(1)]

Leaves: 11

Depth: 5

```
=====
Node
Feature 0 (continuous)
Threshold: 2.9207
n_samples: 200
|- Left (<=):
|   [Node] Feature 2 (categorical)
|   CRIMCOORDS threshold: -0.0027
|   n_samples: 92
|   |- Left (<=):
|   |   [Node] Feature 0 (continuous)
|   |   Threshold: 2.1266
|   |   n_samples: 72
|   |   |- Left (<=):
|   |   |   [LEAF] Predict: 0 | n=49 | dist=(0:49)
|   |   |   |- Right (>):
|   |   |   [Node] Feature 2 (categorical)
|   |   |   CRIMCOORDS threshold: 0.7727
|   |   |   n_samples: 23
|   |   ...
|   |   |- Right (>):
|   |   |   [LEAF] Predict: 1 | n=13 | dist=(0:1, 1:12)
|   |- Right (>):
|   [LEAF] Predict: 1 | n=68 | dist=(1:68)
```

Performance et prédictions du modèle QUEST

Training Accuracy: 93.00%

Sample Predictions:

Index	Actual	Predicted	Correct
0	0	0	Y
1	1	1	Y
2	1	1	Y
3	0	0	Y
4	0	0	Y
5	0	0	Y
6	0	0	Y
7	1	1	Y
8	0	0	Y
9	1	1	Y

Test 1 — Classification binaire (QUEST)

```
=====
TEST 1:  Binary Classification
=====
```

Classes detected: [0 1]

Training samples: 210

Test samples: 90

Accuracy: 0.8889

Classification Report:

	precision	recall	f1-score	support
0	0.93	0.86	0.89	49
1	0.84	0.93	0.88	41
accuracy			0.89	90
macro avg	0.89	0.89	0.89	90
weighted avg	0.89	0.89	0.89	90

Tree Structure:

QUEST Decision Tree Structure:

Classes: [np.int64(0), np.int64(1)]

Leaves: 10

...

```
|- Left (<=):
| [LEAF] Predict: 1 | n=15 | dist=(0:3, 1:12)
|- Right (>):
| [LEAF] Predict: 1 | n=50 | dist=(1:50)
```

TEST 2 — Classification multiclasse (K = 4)

```
=====
TEST 2: Multiclass Classification (K=4)
=====
```

```
Classes detected: [0 1 2 3]
Training samples: 350
Test samples: 150
```

```
Accuracy: 0.6000
```

```
Classification Report:
```

	precision	recall	f1-score	support
0	0.64	0.75	0.69	40
1	0.71	0.39	0.50	31
2	0.78	0.63	0.70	46
3	0.39	0.58	0.46	33
accuracy			0.60	150
macro avg	0.63	0.59	0.59	150
weighted avg	0.64	0.60	0.60	150

```
Tree Structure:
```

```
QUEST Decision Tree Structure:
```

```
...
| |- Right (>):
|   [LEAF] Predict: 1 | n=18 | dist=(0:2, 1:16)
|- Right (>):
   [LEAF] Predict: 1 | n=7 | dist=(0:1, 1:5, 2:1)
```

TEST 3 — Élagage Cost-Complexity

```
=====
TEST 3: Cost-Complexity Pruning
=====
```

```
[Testing different  $ccp_{\alpha}$  values:]
```

```
-----
= 0.00 : leaves = 11,  $train_{acc}$  = 0.9429,  $test_{acc}$  = 0.8889
= 0.01 : leaves = 4,  $train_{acc}$  = 0.9143,  $test_{acc}$  = 0.8000
= 0.02 : leaves = 4,  $train_{acc}$  = 0.9143,  $test_{acc}$  = 0.8000
= 0.05 : leaves = 4,  $train_{acc}$  = 0.9143,  $test_{acc}$  = 0.8000
= 0.10 : leaves = 2,  $train_{acc}$  = 0.7905,  $test_{acc}$  = 0.7333
-----
```

```
Note: Increasing  $\alpha$  leads to more aggressive pruning (fewer leaves)
but may improve test accuracy by reducing overfitting.
```

TEST FINAL — Jeu de données Iris

```
=====
FINAL TEST (Iris dataset) - placed and executed as the last cell
=====
```

```
Classes detected:  [0 1 2]
Training samples:  105
Test samples:      45
```

```
Accuracy (no pruning):  0.8667
```

```
Classification Report (no pruning):
precision recall f1-score support
```

```
0 0.83 1.00 0.91 15
1 0.86 0.80 0.83 15
2 0.92 0.80 0.86 15
```

```
accuracy 0.87 45
macro avg 0.87 0.87 0.86 45
weighted avg 0.87 0.87 0.86 45
```

```
Tree Structure (no pruning):
QUEST Decision Tree Structure:
Classes:  [np.int64(0), np.int64(1), np.int64(2)]
```

```
...
|- Left (<=):
| [LEAF] Predict:  1 | n=33 | dist=(1:31, 2:2)
|- Right (>):
```

```
LEAF
Predict:  2 | n=34 | dist=(1:1, 2:33)
```

```
Output is truncated.  View as a scrollable element or open in a text editor.  Adjust cell
output settings...
```

**TEST SUR DONNÉES RÉELLES — UCI Car
Evaluation**

```
=====
REAL DATA TEST: UCI Car Evaluation
=====
```

```
Loaded Car dataset: 1728 rows, 7 columns
```

```
Sample of dataset and class distribution:
```

```
class
```

```
unacc 1210
```

```
acc 384
```

```
good 69
```

```
vgood 65
```

```
Name: count, dtype: int64
```

```
Classes detected (encoded): [0 1 2 3]
```

```
Training samples: 1209
```

```
Test samples: 519
```

```
Accuracy (Car dataset): 0.9249
```

```
Classification Report:
```

```
precision recall f1-score support
```

```
acc 0.80 0.91 0.85 115
```

```
good 0.93 0.62 0.74 21
```

```
unacc 0.98 0.96 0.97 363
```

```
...
```

```
alpha=-0.000000, leaves=40
```

```
alpha=0.000000, leaves=39
```

```
alpha=0.000000, leaves=38
```

```
alpha=0.000000, leaves=36
```

```
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
```

COMPARAISON QUEST (Python vs R)

```
Fitting QUEST (Python implementation)...
```

```
QUEST (Python) metrics:
```

```
Accuracy: 0.8666666666666667
Balanced Accuracy: 0.8666666666666667
Macro F1: 0.864606657710106
Weighted F1: 0.8646066577101059
Matthews Corr: 0.8041806928957864
Cohen Kappa: 0.8
Confusion Matrix:
[15 0 0
```

```
12 1
```

```
2 12
```

```
]

```

```
Attempting to fit rpart in R (via rpy2)...
```

```
R rpart metrics:
```

```
Accuracy: 0.8888888888888888
Balanced Accuracy: 0.8888888888888889
Macro F1: 0.8887652947719689
Weighted F1: 0.888765294771969
Matthews Corr: 0.8339513040028604
Cohen Kappa: 0.8333333333333334
Confusion Matrix:
[15 0 0
```

```
...
```

```
Matthews Corr: 0.8340
```

```
Cohen Kappa: 0.8333
```

```
Detailed reports are printed above.
```

```
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
```

5. Comprendre QUEST : Théorie et Comparaisons

Pourquoi QUEST est-il non biaisé ?

L'innovation clé de QUEST est la **séparation de la sélection des variables et de la sélection du point de coupure**.

Le problème de biais dans CART et C4.5

Considérons comment CART sélectionne une variable de coupure :

1. Pour chaque variable, tester **tous** les points de coupure possibles
2. Sélectionner la paire (variable, split) avec la meilleure réduction d'impureté

Cela crée un **bias de sélection** car :

- Une variable continue avec de nombreuses valeurs uniques a plus de splits possibles
- Même une variable purement aléatoire pourra sembler bonne par hasard
- Les variables avec beaucoup de catégories ont plus de façons de se partitionner

Exemple : Une variable continue avec 100 valeurs uniques possède 99 splits possibles. Par hasard, certains splits sembleront bons, donnant un avantage injuste par rapport à une variable catégorielle binaire avec un seul split.

Comment QUEST résout ce problème

QUEST utilise une **approche en deux phases** :

1. **Sélection de variable** : Tests d'hypothèse ayant une puissance cohérente entre types de variables
 - ANOVA pour les variables continues (teste les différences de moyenne)
 - Chi-carré pour les variables catégorielles (teste l'indépendance)
 - Ces tests ne dépendent pas du nombre de splits possibles
2. **Sélection du point de coupure** : Pour la variable sélectionnée, utilisation de QDA
 - Méthode paramétrique qui ne recherche pas tous les splits
 - Le point de coupure est calculé analytiquement

Comparaison avec d'autres algorithmes

Aspect	CART	C4.5	QUEST
Sélection de variable	Recherche exhaustive	Recherche exhaustive	Tests statistiques
Sélection de split	Recherche exhaustive	Recherche exhaustive	QDA (analytique)
Biais	Vers variables multi-valeurs	Vers variables multi-catégories	Non biaisé
Vitesse	$O(n \cdot p \cdot m)$	$O(n \cdot p \cdot m)$	$O(n \cdot p)$
Multiway splits	Non (binaire seulement)	Oui	Non (binaire seulement)
Valeurs manquantes	Surrogate splits	Catégorie spéciale	Non géré directement

où : n = nombre d'échantillons, p = nombre de features, m = moyenne de valeurs uniques par feature

Forces de QUEST

1. **Non biaisé** : Pas de biais vers les variables avec beaucoup de splits possibles
2. **Efficace** : $O(n \cdot p)$ par nœud au lieu de $O(n \cdot p \cdot m)$
3. **Interprétable** : Utilise des tests statistiques connus
4. **Principé** : Basé sur la théorie statistique (LDA, ANOVA)

Limitations de QUEST

1. **Classification binaire uniquement** : Conçu pour deux classes
2. **Hypothèse de normalité** : QDA suppose des distributions normales
3. **Splits binaires seulement** : Ne peut pas produire de splits multi-way
4. **Valeurs manquantes** : Non gérées directement dans l'algorithme de base

Quand utiliser QUEST ?

QUEST est particulièrement adapté lorsque :

- Vous avez un mélange de variables catégorielles et continues
- Les variables ont des nombres différents de valeurs uniques
- Vous voulez éviter la sélection de variables aléatoires
- L'interprétation de l'importance des variables est critique
- Vous avez un problème de classification binaire

6. Résumé

Ce notebook a implémenté l'algorithme d'arbre de décision **QUEST** depuis zéro, en suivant l'article original de Loh et Shih (1997).

Composants clés implémentés

- **Algorithme 3 (Sélection de variables)**
 - Test F d'ANOVA pour les variables continues
 - Test du Chi-carré pour les variables catégorielles
 - Test de Levene comme backup
 - Correction de Bonferroni pour tests multiples
- **Algorithme 2 (CRIMCOORDS)**
 - Codage one-hot
 - SVD pour réduction dimensionnelle
 - LDA pour un scoring optimal des catégories
- **Algorithme 1 (QDA Split)**
 - Analyse discriminante quadratique
 - Solution analytique pour le point de coupure
 - Gestion des cas limites

Conclusion Générale

L'étude et l'implémentation de l'algorithme QUEST (Quick, Unbiased, Efficient Statistical Tree) ont permis de comprendre en profondeur les mécanismes d'un arbre de décision non biaisé pour la classification.

- Nous avons exploré la **sélection des variables** via des tests statistiques (ANOVA pour les variables continues, Chi-square pour les catégorielles), éliminant le biais des algorithmes classiques comme CART et C4.5.

- La **transformation des variables catégorielles** avec CRIMCOORDS a permis de projeter les catégories dans un espace numérique optimal, facilitant le calcul des points de coupure.
- Le **calcul des points de split** a été réalisé à l'aide de l'analyse discriminante quadratique (QDA), offrant une solution analytique rapide et efficace.
- L'**élagage cost-complexity** a été appliqué pour régulariser l'arbre, garantissant un compromis optimal entre complexité et performance.
- Des **tests sur des jeux de données synthétiques et réels** (Iris, UCI Car Evaluation) ont montré que QUEST offre une précision élevée, une interprétabilité claire et une robustesse face aux données mixtes.
- La comparaison avec d'autres méthodes a mis en évidence les **forces** de QUEST : absence de biais de sélection, efficacité de calcul et interprétabilité, tout en notant certaines **limitations** (classification binaire de base, hypothèse de normalité pour QDA, splits binaires uniquement).

En résumé, QUEST constitue une approche statistique rigoureuse pour la construction d'arbres de décision, particulièrement adaptée aux jeux de données combinant variables continues et catégorielles, et lorsque la neutralité dans le choix des variables est essentielle pour l'analyse.

Références

- Loh, W.-Y., & Shih, Y.-S. (1997). Split selection methods for classification trees. *Statistica Sinica*, 7, 815-840.
- Loh, W.-Y. (2014). Fifty years of classification and regression trees. *International Statistical Review*, 82(3), 329-348.
- Research Paper: <https://www3.stat.sinica.edu.tw/statistica/oldpdf/A7n41.pdf>
- Document of QUEST Algorithm: <https://public.dhe.ibm.com/software/analytics/spss/support/StatQUEST.pdf>
- Algorithm Implementation with C/C++: <https://github.com/hetianle/QuestDecisionTree>